A Genetic Algorithm Framework for Optimizing Three-Impulse Orbital Transfers with Poliastro Simulation - Revisitado para PSO

Inteligência Computacional - PSO Híbrido

Pedro de Araujo Ribeiro
Arthur Henrique de Souza
Thais Damasceno Silva

Problema - Revisado

Devemos levar um objeto de uma órbita A para uma órbita B, este processo gasta combustível.

O trajeto percorrido e combustível gastos dependem ((simplificado)) do raio *rf* da órbita de partida, e do raio *rb* da trajetória planejada.

Ao alterar a trajetória planejada, podemos calcular rotas com duração de tempo e consumo de combustíveis diferentes.

O objetivo será buscar uma rota que minimiza o consumo de combustível.

Há uma solução analítica chamada transferência de Homman, mas ela não produz a trajetória mais eficiente em relação ao combustível.

Metodologia - PSO Híbrido

Representação: Cada partícula é representada por um número real *rho* referente a proporção de *rb*/*rf* (raio da trajetória sobre raio do ponto de partida).

Por ser um número real, rho será forçado dentro de um intervalo específico.

No código, nossa população de partículas é um vetor de *rho*s.

Função de fitness: Minimizar o gasto de combustível*, em função de *rho*.

Quando rho menor que 1 ou maior que o limite, uma punição massiva é aplicada e o resultado da equação é transformado em 10^6.

Fórmula de velocidade: $v(t+1) = w*v(t) + c1*r1*(pbest - x) + c2*r2*(lbest - x)$

Atualização da posição: $x(t+1) = x(t) + v(t+1)$

Metodologia - PSO Híbrido

Parâmetros:

Número de partículas: 40

Peso da inércia (ω): 0.7

Coeficiente cognitivo (c1): 1.5

Coeficiente social (c2): 1.5

Iterações: 70

Intervalo r1 e r2: [0,1)

“Best”: *local_best* por vizinhança, considera os dois vizinhos de índice da partícula.

Hibridização: Mutação simples onde o valor da velocidade da partícula é multiplicado por um número entre 0.5 e 1.5. Possui 10% de chance de ocorrer depois do cálculo da nova velocidade e antes da atualização da posição.

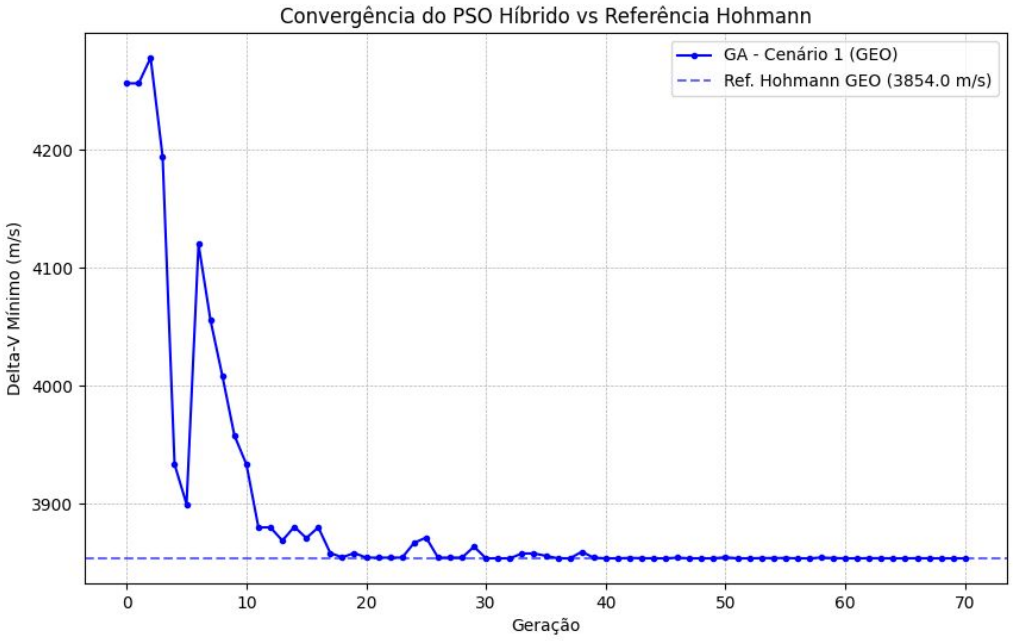
Resultados - Artigo

O artigo original contempla apenas uma implementação de um Algoritmo Genético e o compara com a solução analítica, aqui estão os resultados:

Metric	Scenario 1: LEO to GEO	Scenario 2: LEO to Far Orbit
GA-Optimized Solution		
Intermediate Radius (r_b) [km]	42,164.00	118,190,508.83
Total ΔV [m/s]	3853.96	3887.15
Total Time [days]	0.22	52,374.31
Hohmann Transfer (for comparison)		
Total ΔV [m/s]	3853.96	4100.62
Total Time [days]	0.22	1.09

Scenario	Optimal Strategy Found by GA	ΔV Savings (m/s)
LEO to GEO ($r_f/r_i \approx 6.2$)	Hohmann Transfer	0.00
LEO to Far Orbit ($r_f/r_i = 20$)	Bi-elliptic Transfer	213.47

Resultados - Nossa Implementação



Intervalo de restrição [1,40]

Melhor rho encontrado: 1.000008

Melhor rb encontrado: 42164.33 km

Delta-V PSO: 3853.9624 m/s

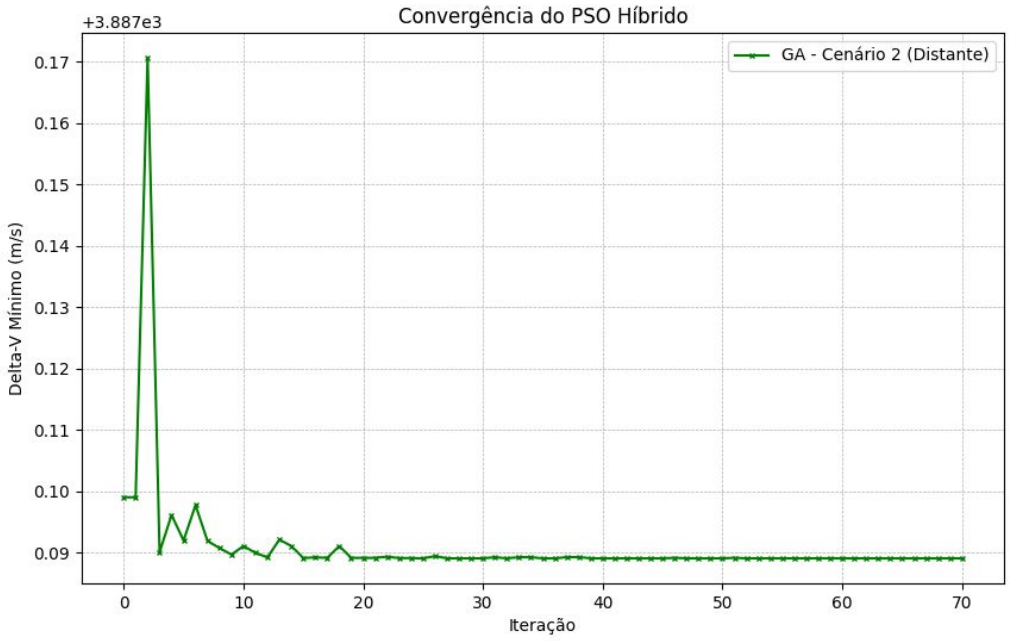
Duração PSO: 0.7191 dias

Delta-V Hohmann: 3853.96 m/s

Duração Hohmann: 0.22 dias

Diferença (Economia): 0.00 m/s

Resultados - Nossa Implementação



Intervalo de restrição [1,1000]

Melhor rho encontrado: 999.999669

Melhor rb encontrado: 42163986.05 km

Delta-V GA: 3887.0891 m/s

Duração GA: 11159.4486 dias

Delta-V Hohmann: 4100.62 m/s

Duração Hohmann: 1.09 dias

Diferença (Economia): 213.53 m/s

Comparação

	LEO-GEO original	LEO-GEO AG	LEO-GEO PSO	LEO-Distante original	LEO-Distante AG	LEO-Distante PSO
rb (Km)	42,164.00	42,164.17	42,164.33	118,190,508.83	135,277,472.06	135,562,682.96
Delta V (m/s)	3853.96	3853.96	3853.9624	3887.15	3887.09	3887.0891
Duração (dias)	0.22*	0.72	0.7191	52,374.31*	64,125.85	64,328.6492

Comparação - PSO e AG por iteração

Iteração/rho	LEO-GEO AG	LEO-GEO PSO	LEO-Distante AG	LEO-Distante PSO
1	1.9436	2.708347	962.8165	968.676190
5	2.1449	1.691087	998.7229	997.362789
10	1.0021	1.069330	999.9045	998.408427
25	1.0002	1.007987	999.9950	998.895093
40	1.0000	1.011259	993.3575	999.993909
55	1.0000	1.000686	999.9995	999.974719
70	1.0000	1.000087	1000.0000	999.999809

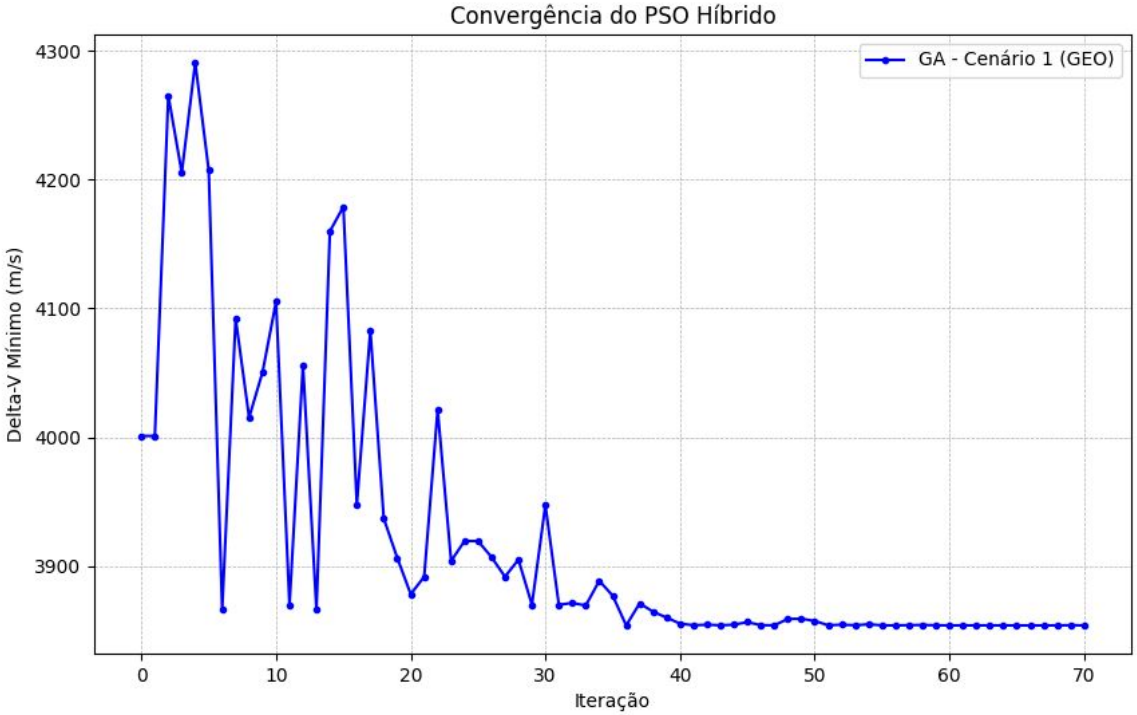
Comparação - Variação de *rho* em PSO vs ganho real

Iteração	rho LEO-GEO	consumo LEO-GEO	rho LEO-Distante	consumo LEO-Distante
1	2.708347	4256.177450	968.676190	3887.101939
5	1.691087	4120.395222	997.362789	3887.090104
10	1.069330	3897.036681	998.408427	3887.089686
25	1.007987	3859.231820	998.895093	3887.089491
40	1.011259	3861.367879	999.993909	3887.089053
55	1.000686	3854.413642	999.974719	3887.089061
70	1.000087	3854.015128	999.999809	3887.089051

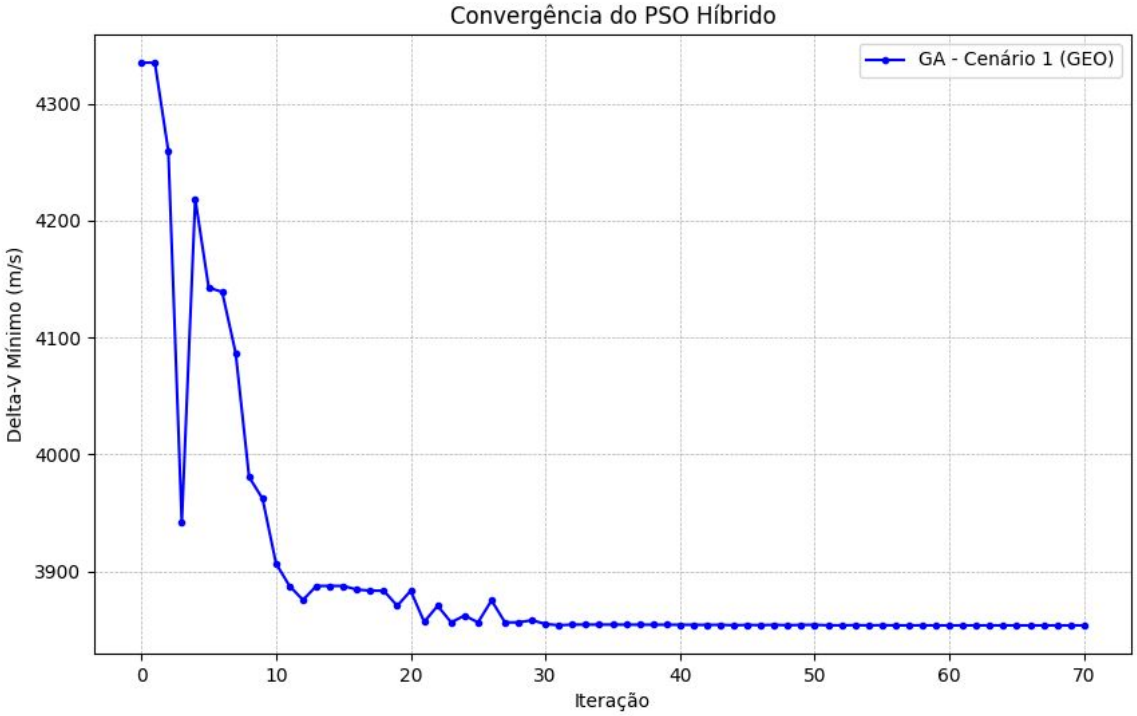
Comparação - Alterando parâmetros do PSO

LEO-GEO Iteração/rho	Controle	c1 = 3	c1 = 0	c2 = 3	c2 = 0	w = 0.1	w = 1
1	2.708347	1.286774	4.336903	1.827963	2.424875	1.793283	1.811594
5	1.691087	2.217006	1.799112	3.387743	1.380687	1.102623	1.306922
10	1.069330	1.625299	1.086522	1.012022	1.033312	1.004921	1.455404
25	1.007987	1.109408	1.003930	1.055448	1.003557	1.000003	1.066871
40	1.011259	1.002119	1.000855	1.055721	1.000025	1.000000	3.525462
55	1.000686	1.000121	1.000301	1.001885	1.000833	1.000000	2.303704
70	1.000087	1.000070	1.000003	1.004770	1.000142	1.000000	1.003364

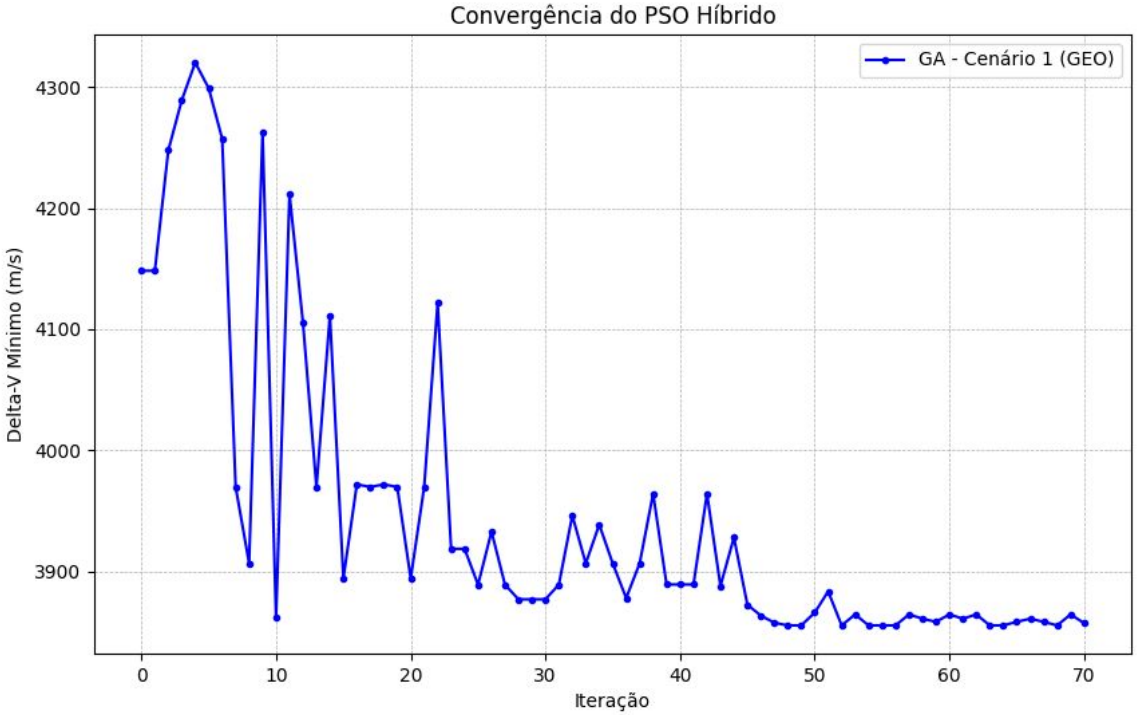
Comparação - c1 = 3



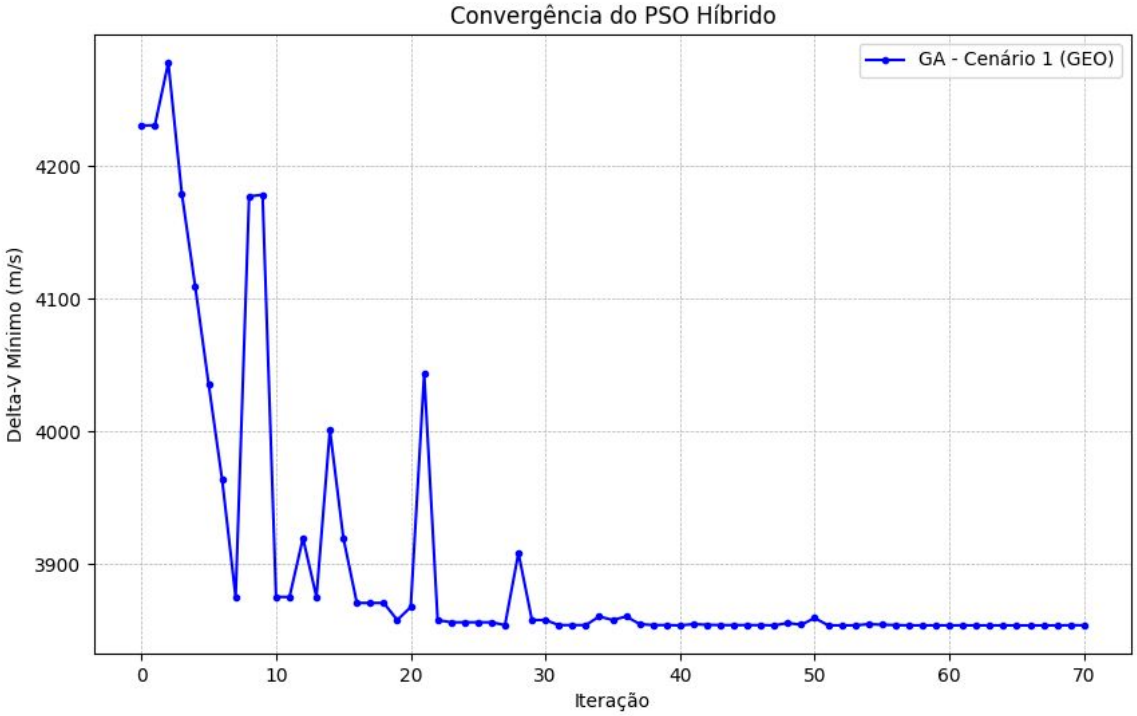
Comparação - c1 = 0



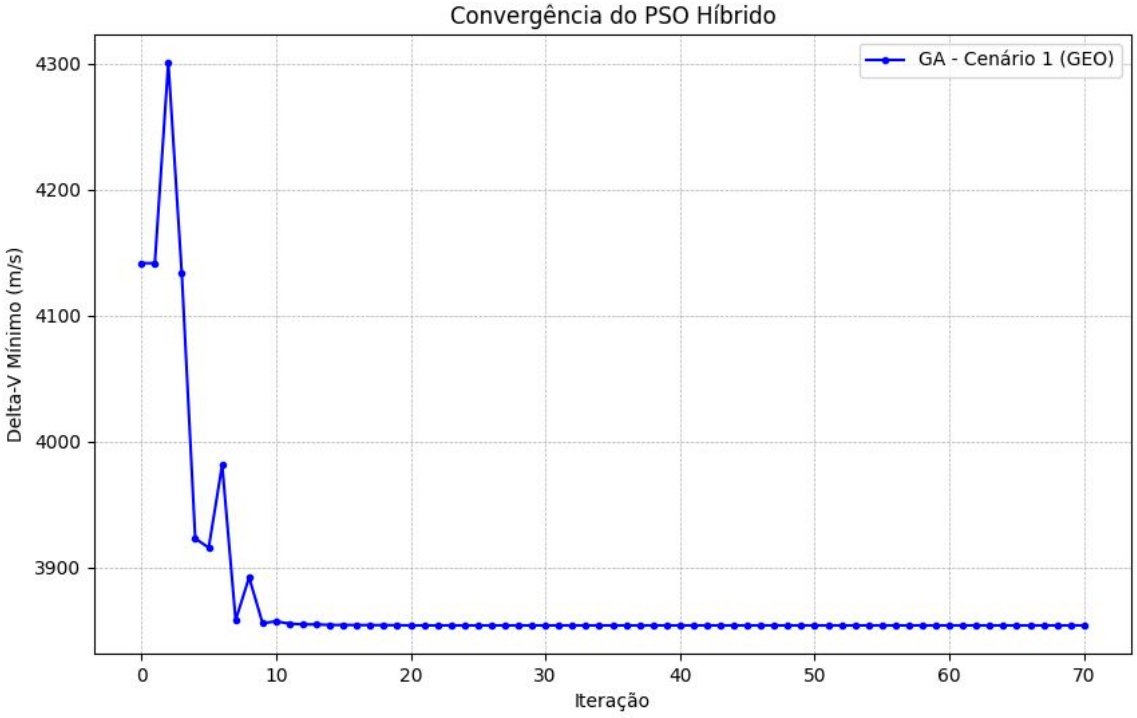
Comparação - c2 = 3



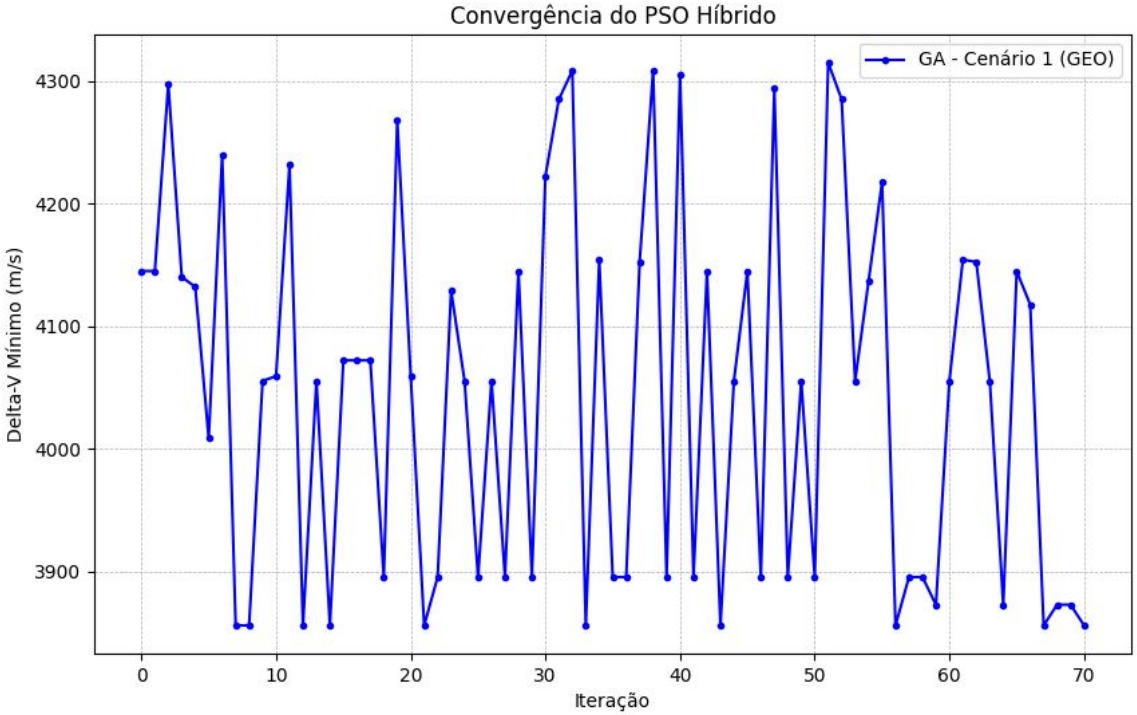
Comparação - c2 = 0



Comparação - w = 0.1



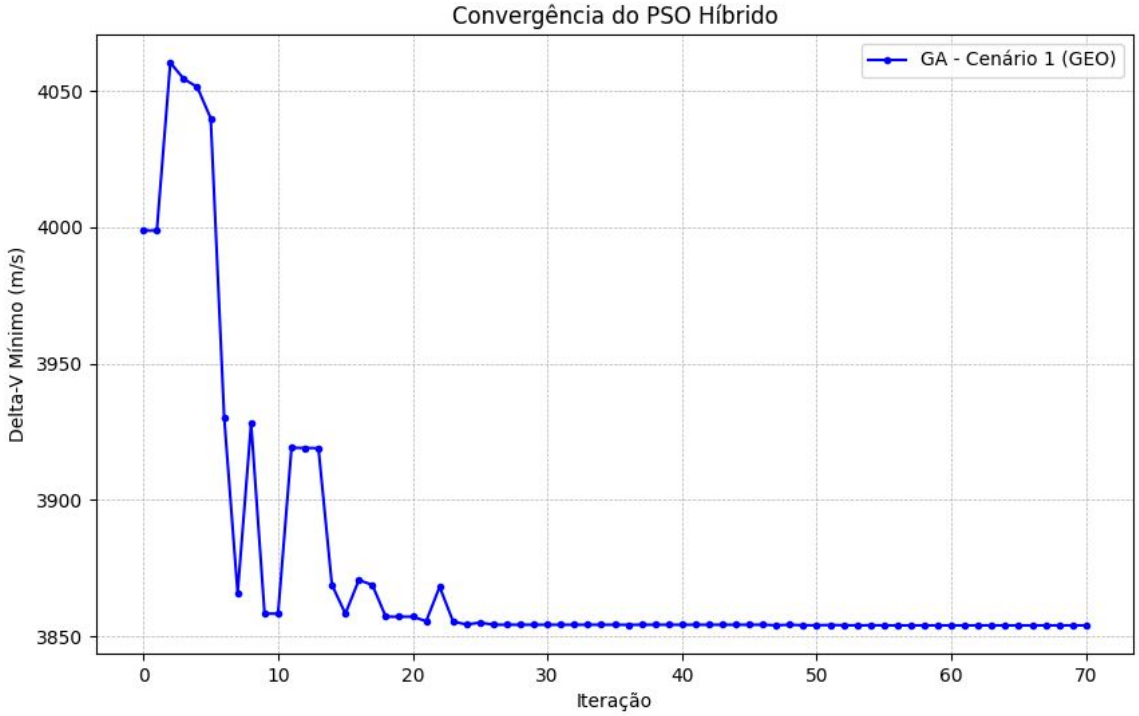
Comparação - w = 1



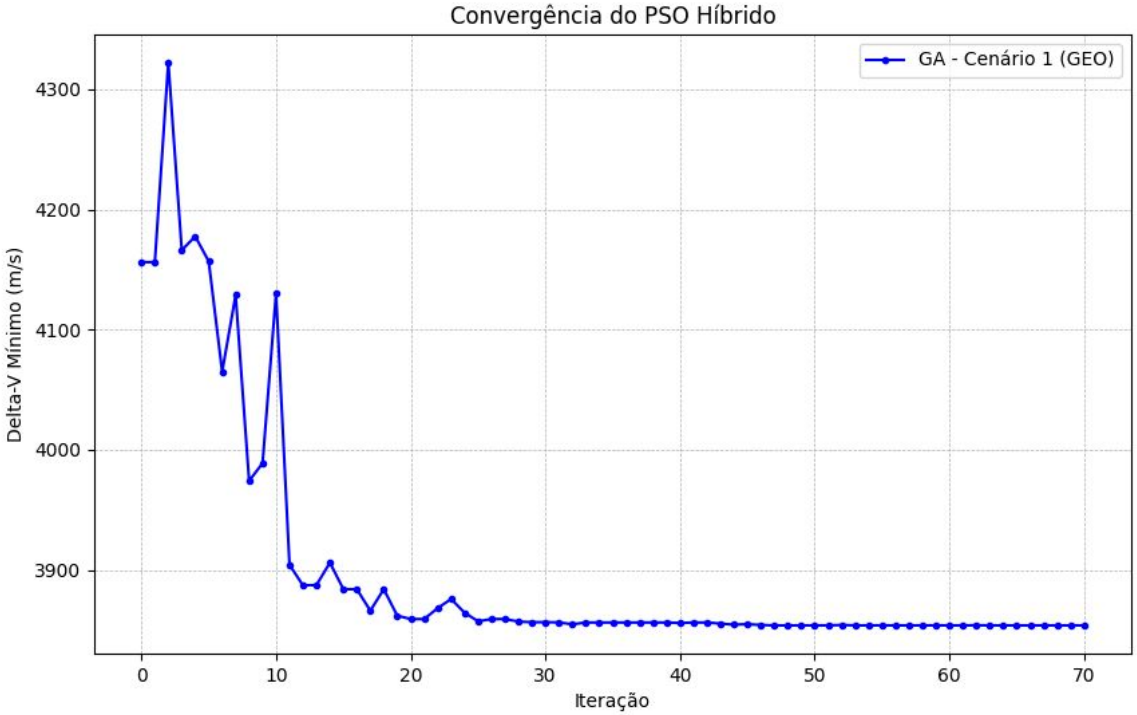
Comparação - Alterando parâmetros do PSO

LEO-GEO Iteração/rho	Controle	Sem mutação	60% de mutação	Global best	“Handicap”
1	2.708347	1.281207	1.871625	2.185158	30.143544
5	1.691087	1.394533	1.875527	1.470993	3.411057
10	1.069330	1.006532	1.736122	1.008607	1.335211
25	1.007987	1.001574	1.005231	1.001904	1.002012
40	1.011259	1.000375	1.003104	1.002939	1.000012
55	1.000686	1.000022	1.000113	1.000003	1.000032
70	1.000087	1.000000	1.000029	1.000002	1.000004

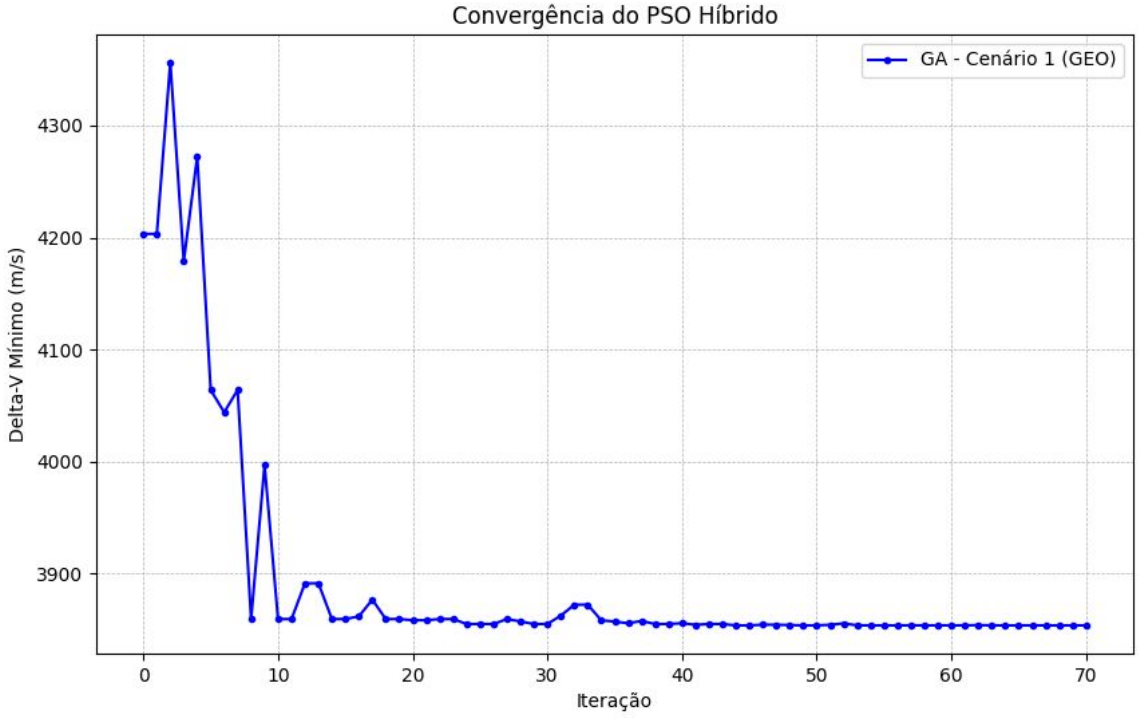
Comparação - Sem mutação



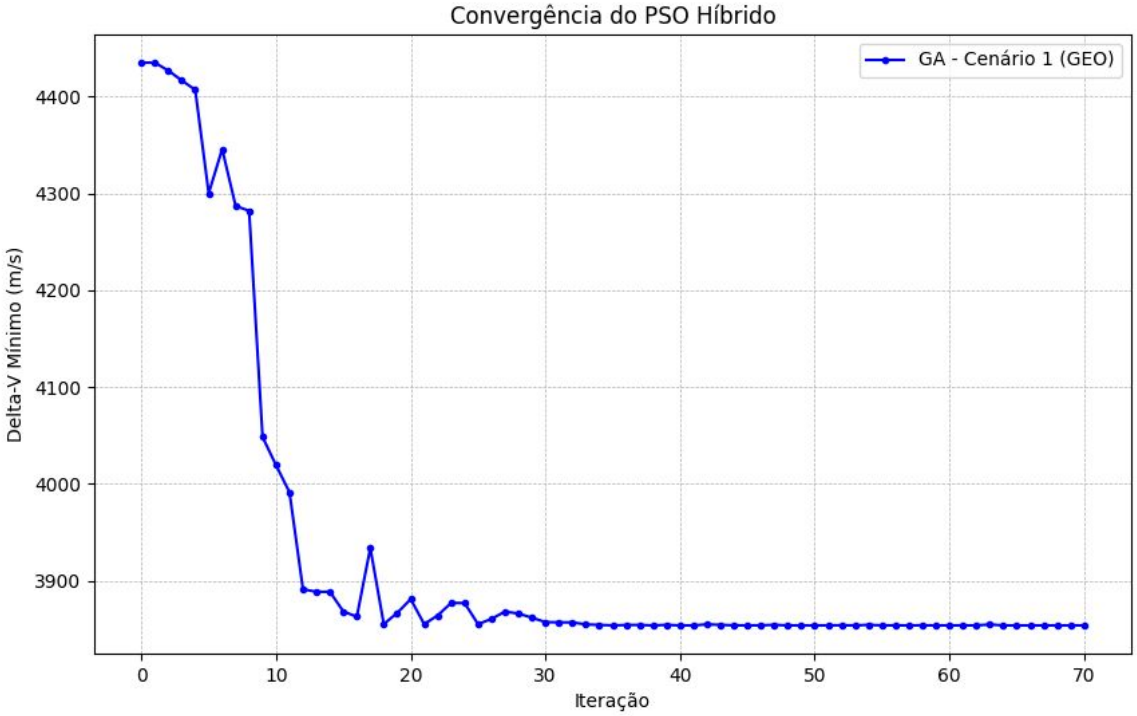
Comparação - 60% de chance de mutação



Comparação - Global best



Comparação - “Handicap” ou desvantagem



Conclusão

PSO, por sua natureza, estará obrigatoriamente sempre em movimento. Em um problema com mais de uma solução ideal, ou soluções ideais próximas umas das outras, a ausência de medidas de parada como “ganho por iteração” pode levar as partículas a ficarem “dançando” entre pontos ideais.

No problema abordado, *exploration* era mais importante que *exploitation* pois estamos explorando um universo de soluções real em um intervalo linear onde há diversas soluções ótimas. Para melhorar a exploration utilizamos de parâmetros como a inclusão de mutações e uso de *local_best* por vizinhança.

Também observamos o impacto causado pela população inicial, em especial quando esta inclui uma solução muito próxima de uma ideal.

Referências e código

Artigo original: Arxiv

Código: Github

Obrigado pela atenção